

Instituto Superior de Engenharia de Coimbra

Programação WEB



Trabalho Prático
Licenciatura Engenharia Informática 2025 / 2026

Duarte Xavier de Oliveira Santos

a2022149622@isec.pt

Gustavo Trigo Costa

a2023145800@isec.pt

1. Introdução.....	2
2. Arquitetura da Solução.....	3
2.1 Tecnologias Utilizadas.....	4
2.2 Estrutura dos Projetos.....	4
2.3 Fluxo de Dados e Reutilização.....	6
3. Modelo de Dados.....	6
3.1 Diagrama Entidade-Relacionamento.....	6
3.2 Descrição das Entidades Principais.....	8
4. Implementação e Funcionalidades.....	10
4.1 Navegação por Categorias e Filtragem Dinâmica.....	10
4.2 Gestão do Carrinho de Compras (CarrinhoService).....	11
4.3 Autenticação e Segurança.....	12
4.4 Funcionalidades de Cliente (Histórico de Encomendas).....	13
4.5 Funcionalidades de Fornecedor (Monitorização de Vendas).....	14
4.6 Versão Mobile (.NET MAUI).....	15
5. Desafios e Soluções.....	16
6. Conclusão.....	17

1. Introdução

O principal objetivo deste trabalho prático, realizado no âmbito da unidade curricular de Programação Web (PWEB), foi conceber e implementar uma arquitetura completa baseada no ecossistema **.NET 8**, separando a lógica de negócio (*Backend*) das diversas interfaces de utilizador.

O projeto visa demonstrar competências na criação de APIs RESTful, no desenvolvimento de interfaces web distintas para o público e para a administração (*Backoffice*), e na integração com aplicações móveis híbridas.

2. Arquitetura da Solução

A solução **MyCOLL** foi desenvolvida seguindo uma arquitetura multicamada e distribuída, baseada integralmente no ecossistema .NET 8. A arquitetura foi desenhada para garantir uma clara separação de responsabilidades (*SoC - Separation of Concerns*), onde a lógica de acesso a dados e regras de negócio reside no servidor (*Backend*), enquanto a interface e experiência de utilizador são geridas pelas aplicações cliente (*Frontend Web e Mobile*).

2.1 Tecnologias Utilizadas

Para a concretização dos requisitos, foram seleccionadas as seguintes tecnologias:

- **Backend:** ASP.NET Core Web API, Entity Framework Core (ORM) e SQL Server.
- **Frontend Web:** Blazor WebAssembly (WASM).
- **Frontend Mobile:** .NET MAUI (Multi-platform App UI) com Blazor Hybrid.
- **Linguagem:** C# e Razor (HTML/CSS).

2.2 Estrutura dos Projetos

A solução Visual Studio encontra-se organizada em cinco projetos distintos, cada um com um papel específico no sistema:

- **"MyCOLL.API" (Backend / O "Cérebro"):** Este é o projeto central do servidor. É responsável por expor os *endpoints* REST que servem os dados às aplicações cliente. Gere a comunicação com a base de dados SQL Server através do Entity Framework Core e implementa a segurança do sistema, incluindo a autenticação e autorização via JWT (*JSON Web Tokens*) e ASP.NET Core Identity. É também responsável por servir os ficheiros estáticos (imagens dos produtos) para a Web e Mobile.
- **"MyCOLL.Shared" (Biblioteca de Classes):** Este projeto contém as classes que são comuns a toda a solução, nomeadamente os Modelos de dados (ex: **Produto**, **Categoria**, **Encomenda**) e os DTOs (*Data Transfer Objects*). A sua existência elimina a duplicação de código, garantindo que o *Backend* e o *Frontend* "falam a mesma língua" e partilham as mesmas validações de dados.
- **"MyCOLL.RCL" (Razor Class Library - Interface Partilhada):** Este é o componente arquitetural mais importante do *Frontend*. Em vez de duplicar as páginas e componentes visuais entre o site e a app móvel, toda a interface de utilizador (Páginas: *Home*, *Carrinho*, *Login*, *DetalhesProduto*) reside nesta biblioteca. Isto permite uma reutilização de código próxima dos 100%: qualquer alteração visual feita na RCL reflete-se automaticamente tanto na Web como no Android.
- **"MyCOLL.GestaoLoja" (Backoffice / Administração):** Este projeto implementa a interface de gestão reservada, desenvolvida em Blazor Server (tecnologia recomendada para acesso direto). O seu acesso é exclusivo a Administradores e Funcionários. Ao contrário da aplicação pública, esta aplicação **interage diretamente com a Base de Dados e**

não através da API, permitindo a gestão de produtos, stocks, utilizadores e encomendas com maior performance e segurança interna.

- **"MyCOLL.Public" (Cliente Web):** Trata-se do projeto Blazor WebAssembly que corre diretamente no *browser* do cliente. Este projeto funciona como um "hospedeiro" para a RCL, carregando a aplicação num ambiente web tradicional.
- **MyCOLL.Public.Maui (Cliente Mobile):** O contentor nativo para dispositivos móveis (Android). Utiliza a tecnologia **Blazor Hybrid** através de um componente "BlazorWebView". Este componente permite renderizar a interface definida na RCL dentro de uma aplicação Android nativa, com acesso às capacidades do dispositivo, sem necessidade de reescrever o código em XAML.

2.3 Fluxo de Dados e Reutilização

A arquitetura da solução foi desenhada com dois fluxos de dados distintos, adequados à natureza de cada aplicação:

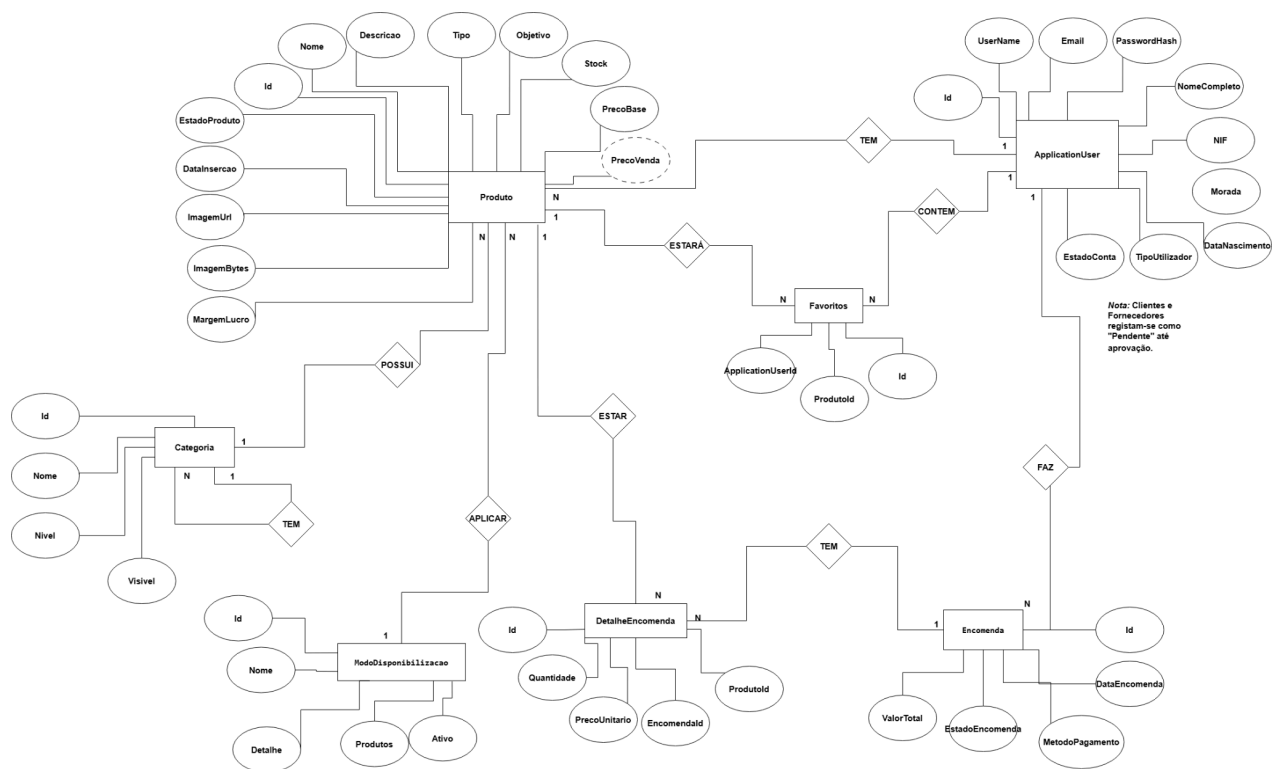
- **Frontend Público (Web e Mobile):** Segue uma arquitetura distribuída, onde a API ("MyCOLL.API") funciona como a única fornecedora de dados. O processamento visual ocorre localmente no dispositivo do utilizador (Cliente), garantindo uma separação total entre a lógica de negócio e a apresentação.
- **Backoffice (Gestão de Loja):** Sendo uma aplicação de uso interno e seguro (Blazor Server), comunica diretamente com a Base de Dados SQL Server via Entity Framework Core. Esta abordagem elimina a latência de chamadas HTTP e permite uma gestão de dados mais performante para tarefas administrativas complexas, sem passar pela API pública.

A reutilização de código foi maximizada através da biblioteca partilhada "MyCOLL.RCL". Esta estratégia permitiu centralizar componentes visuais, serviços de comunicação e a gestão de estado (ex: carrinho), assegurando que a mesma lógica em C# alimenta simultaneamente as versões WebAssembly e MAUI do frontend público, reduzindo drasticamente a duplicação de esforço..

3. Modelo de Dados

3.1 Diagrama Entidade-Relacionamento

A figura abaixo ilustra a estrutura das tabelas e os relacionamentos entre as entidades principais do sistema.



3.2 Descrição das Entidades Principais

Para suportar os requisitos funcionais da loja e do backoffice, destacam-se as seguintes estruturas de dados:

- **Ecossistema de Identidade (AspNetUsers e tabelas relacionadas):** A gestão de autenticação baseia-se no sistema padrão ASP.NET Core Identity. A tabela **AspNetUsers** armazena as informações críticas de acesso (email, *hash* da password, dados de segurança). A distinção de permissões é feita através da relação com a tabela **AspNetRoles**, permitindo classificar os utilizadores em três perfis distintos: **Administrador** (gestão total), **Fornecedor** (gestão de stock) e **Cliente** (compras)
- **Catálogo (Produtos e Categorias):**
 - **Produtos:** Entidade central que armazena os detalhes dos artigos colecionáveis (Nome, Descrição, Preço, Stock, imageUrl). Possui uma chave estrangeira para a **Categoria**.
 - **Categorias (Estrutura Recursiva):** Esta tabela possui uma particularidade arquitetural fundamental para o projeto: uma relação de auto-referência (recursividade). O campo "CategoriaPaiId" aponta para a própria tabela **Categorias**. Isto permite criar uma hierarquia de árvore com profundidade ilimitada (ex: Numismática [Pai] -> Moedas Euro [Filho] -> Portugal [Neto]), suportando assim a navegação por "frisos deslizantes" de 3 níveis implementada no *Frontend*.
- **Vendas (Encomendas e DetalhesEncomenda):** A estrutura de vendas segue o padrão mestre-detelhe:
 - **Encomendas:** Representa o "cabeçalho" da transação, armazenando quem fez o pedido (**ClienteId**), quando (**Data**), o valor total e o **Estado** da encomenda (Pendente, Paga, Enviada, Entregue, Cancelada), essencial para o acompanhamento no *Backoffice* e na Área de Cliente.

- **DetalhesEncomenda:** Armazena as linhas individuais da encomenda. É crucial notar que esta tabela guarda o **PreçoUnitario** no momento da compra. Isto garante a integridade histórica dos dados financeiros, mesmo que o preço do produto original seja alterado no futuro.

4. Implementação e Funcionalidades

Neste capítulo são detalhadas as principais funcionalidades desenvolvidas, com foco nas soluções técnicas adotadas para resolver os requisitos de usabilidade e gestão de estado da aplicação **MyCOLL**.

4.1 Navegação por Categorias e Filtragem Dinâmica

Para facilitar a navegação num catálogo extenso de colecionáveis, foi implementado um sistema de filtragem hierárquica visualizado através de "frisos deslizantes" (*sliders*). O utilizador pode afunilar a pesquisa em três níveis: Categoria Principal (ex: Numismática), Sub-Categoria (ex: Portugal) e Específica (ex: Monarquia).

Um dos desafios técnicos nesta implementação foi garantir que, ao seleccionar uma categoria "pai" (ex: Moedas), o sistema apresentasse não só os produtos directamente associados a ela, mas também todos os produtos das suas sub-categorias (filhas e netas). Para tal, foi implementado um algoritmo de **recursividade** em C#. O método “ObterIdsRecursivos” percorre a árvore de categorias e recolhe todos os IDs descendentes, permitindo filtrar a lista de produtos de forma abrangente e intuitiva.

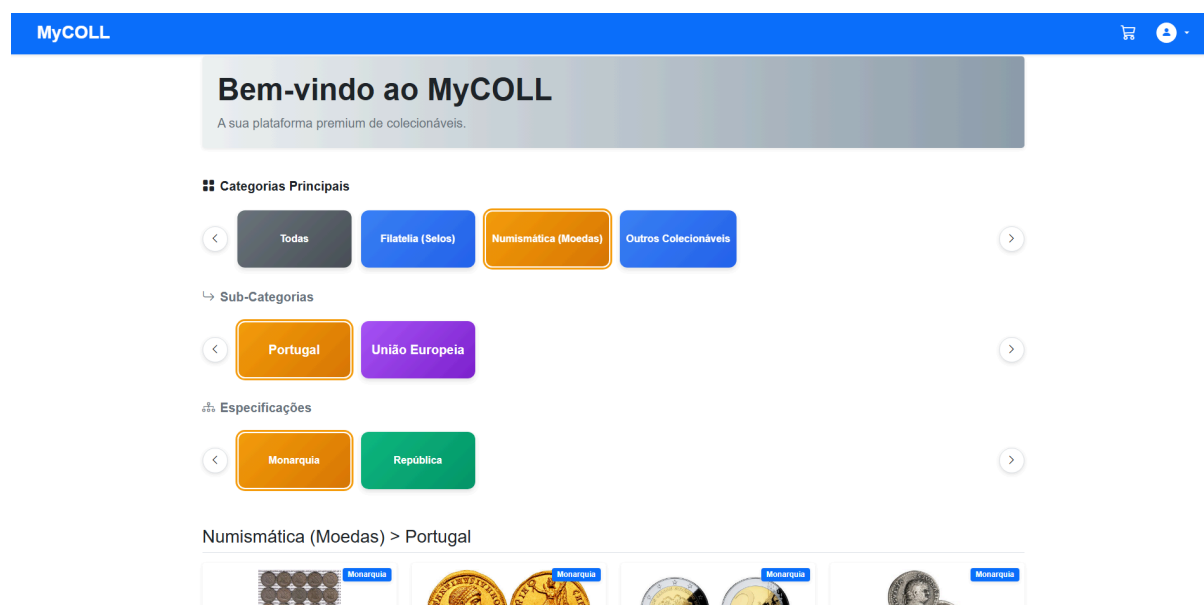


Fig 1. Página inicial com navegação por categorias em níveis.

4.2 Gestão do Carrinho de Compras (CarrinhoService)

A gestão do estado de compras no lado do cliente (*Frontend*) é assegurada pelo serviço "CarrinhoService". Este serviço atua como um *Singleton* durante a sessão do utilizador e implementa duas características fundamentais:

- **Persistência de Dados:** Recorrendo à biblioteca "Blazored.LocalStorage", o conteúdo do carrinho é guardado no *browser* do utilizador. Isto garante que, mesmo que a página seja fechada ou recarregada (*refresh*), os itens seleccionados não são perdidos, melhorando significativamente a experiência de utilização.
- **Reatividade em Tempo Real:** O serviço utiliza um sistema de eventos C# ("OnChange"). Sempre que um produto é adicionado ou removido, o serviço dispara um evento que notifica a barra de navegação ("MainLayout"), atualizando instantaneamente o *Badge* vermelho com a contagem de itens, sem necessidade de recarregar a página.

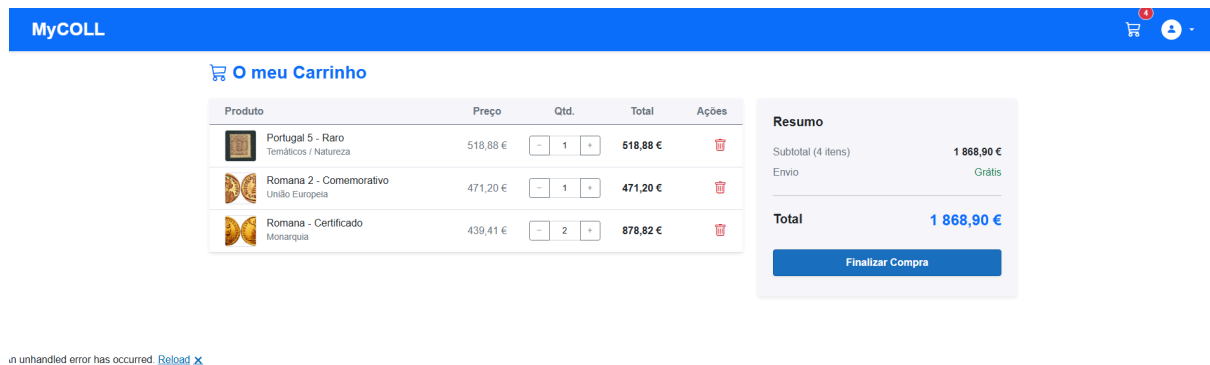


Fig 2. Interface do Carrinho de Compras com cálculo de totais.

4.3 Autenticação e Segurança

O sistema adota uma abordagem mista de segurança. Utilizadores anónimos (não autenticados) possuem acesso de leitura ao catálogo e podem adicionar itens ao carrinho. No entanto, o processo de finalização de compra (*Checkout*) é restrito.

A segurança foi implementada a dois níveis:

- **Interface (UI):** O componente "AuthorizeView" é utilizado para esconder ou mostrar botões sensíveis (como "Finalizar Compra" ou acesso ao *Backoffice*) dependendo do estado de autenticação.
- **Rotas ("App.razor"):** Foi configurada a proteção global de rotas. O atributo "[Authorize]" impede que utilizadores não autenticados acedam diretamente a URLs protegidos (ex: "/encomendas/meuspedidos"), redirecionando-os automaticamente para a página de Login.

4.4 Funcionalidades de Cliente (Histórico de Encomendas)

Para a visualização do histórico de compras na "Área de Cliente", optou-se por uma interface baseada no componente *Accordion* do *Bootstrap*. Esta escolha de *design* é particularmente relevante para a versão móvel, pois permite apresentar uma lista compacta e limpa das encomendas (mostrando apenas a data, valor e estado). Ao clicar numa encomenda, o painel expande-se para revelar os detalhes dos produtos adquiridos, otimizando o espaço de ecrã disponível.

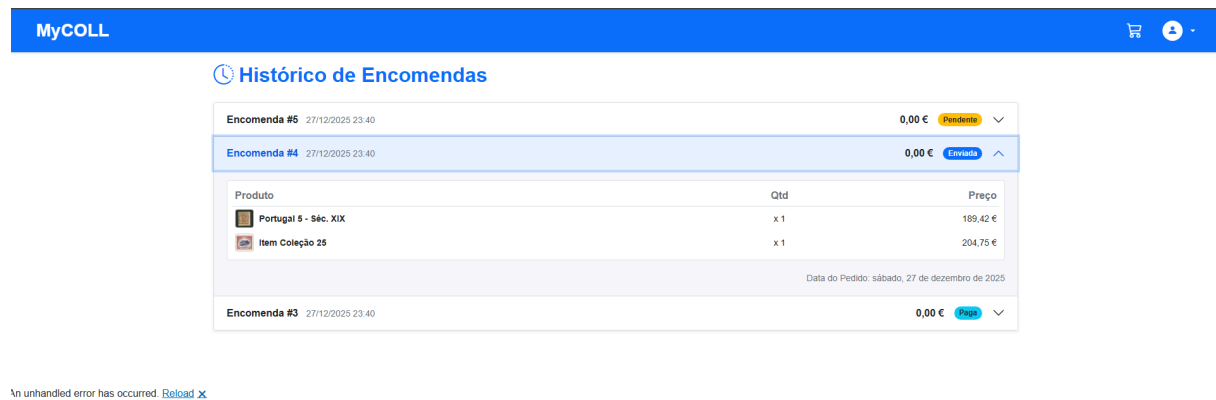


Fig 3. Histórico de encomendas com visualização expansível.

4.5 Funcionalidades de Fornecedor (Monitorização de Vendas)

Foi implementada uma área dedicada aos utilizadores com perfil de **Fornecedor**. Através das página "Gerir Vendas" e "Gerir Produtos", o fornecedor tem acesso a um *dashboard* personalizado onde pode consultar o estado de todos os produtos que colocou na plataforma.

Tecnicamente, esta funcionalidade reaproveita a autenticação para filtrar a base de dados de produtos pelo ID do utilizador logado. A interface permite distinguir rapidamente entre:

- **Artigos Disponíveis:** Produtos que ainda se encontram em *stock* e visíveis na loja pública.
- **Artigos Vendidos:** Produtos que já foram associados a uma encomenda paga.

Esta funcionalidade é crucial para a transparência do negócio, permitindo aos parceiros da MyCOLL saberem exatamente quando os seus artigos foram comercializados.

MyCOLL						
Minhas Vendas						Total Faturado 1 637,34 €
Data	Produto	Qtd	Valor	Estado	Ações	
27/12/2025 23:40	Item Coleção 28 Ref: #28	2 x	108,42 €	Pendente	Detalhes	
27/12/2025 23:40	Portugal 5 - Séc. XIX Ref: #1	1 x	189,42 €	Enviada	Detalhes	
27/12/2025 23:40	Portugal 5 - Séc. XIX Ref: #1	1 x	189,42 €	Paga	Detalhes	
27/12/2025 23:40	Aleatorias - Séc. XIX Ref: #5	1 x	441,78 €	Paga	Detalhes	
27/12/2025 23:39	Portugal 5 - Raro Ref: #2	1 x	518,88 €	Entregue	Detalhes	
22/12/2025 23:35	Portugal 5 - Séc. XIX Ref: #1	1 x	189,42 €	Enviada	Detalhes	

Unhandled error has occurred. [Reload](#)

Fig 5. Painel de controlo do fornecedor com listagem de artigos e respetivo estado.

4.6 Versão Mobile (.NET MAUI)

A versão móvel do MyCOLL não foi reescrita de raiz, mas sim integrada através do **.NET MAUI Blazor Hybrid**. O projeto MAUI atua como um contentor nativo que utiliza o componente "BlazorWebView" para renderizar as páginas definidas na biblioteca partilhada ("MyCOLL.RCL").

Para garantir o funcionamento correto em ambiente Android, foram necessárias configurações específicas ao nível do sistema operativo, nomeadamente no ficheiro "AndroidManifest.xml". Foram explicitamente declaradas as permissões "ACCESS_NETWORK_STATE" e "INTERNET" para permitir a comunicação da aplicação com a API externa, bem como a configuração de tráfego HTTP para ambiente de desenvolvimento (ou uso de *Dev Tunnels*).

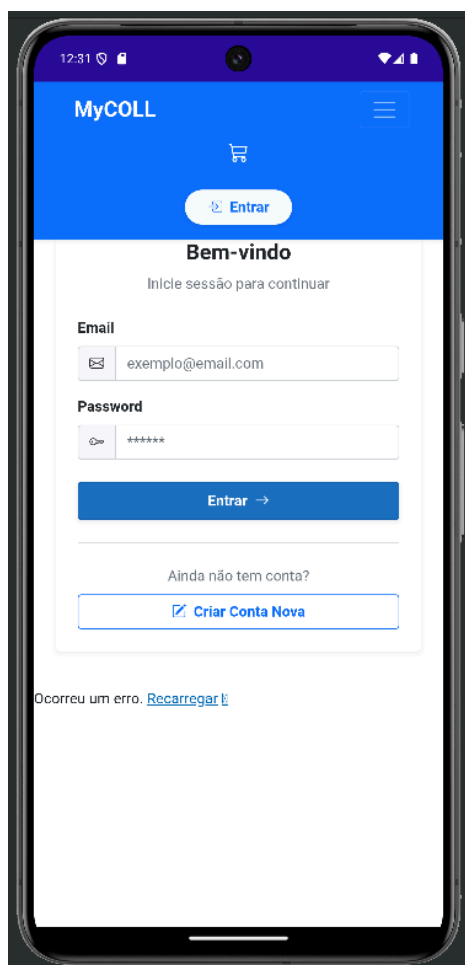


Fig 4. Aplicação a correr em emulador Android (Pixel 5), reutilizando a interface Web.

5. Desafios e Soluções

- **Desafio 1: Conectividade Android e Bloqueio SSL (Localhost)**

- **Problema:** A comunicação entre o emulador Android e a API local falhava devido a dois fatores: o isolamento de rede do emulador (onde localhost refere-se ao próprio dispositivo e não ao computador anfitrião) e a política de segurança do Android que bloqueia, por defeito, tráfego HTTP não encriptado (Cleartext) e rejeita certificados SSL de desenvolvimento autoassinados.
- **Solução:** Utilização de **Dev Tunnels** no Visual Studio para expor a API através de um endereço público acessível pelo emulador, contornando as restrições de IP local. Complementarmente, foi configurado o AndroidManifest.xml (android:usesCleartextTraffic="true") para autorizar a comunicação durante a fase de desenvolvimento.

6. Conclusão

A realização do projeto **MyCOLL** foi fundamental para consolidar conhecimentos avançados sobre o desenvolvimento de aplicações distribuídas no ecossistema **.NET 8**.

A principal aprendizagem deste trabalho foi a aplicação prática do princípio de **Separação de Responsabilidades**. Ao desacoplar totalmente o Backend das interfaces de cliente, compreendemos a importância de construir sistemas modulares e escaláveis. A utilização de **Blazor Hybrid** e da **RCL** destacou-se como uma competência valiosa, provando ser possível reaproveitar mais de 90% do código **C#** entre ambientes Web e Mobile, otimizando drasticamente o desenvolvimento.

O projeto permitiu ainda superar desafios técnicos reais, como a configuração de redes em emuladores Android (Dev Tunnels) e a distinção entre modelos de execução do lado do cliente (*WebAssembly*) versus lado do servidor (*Blazor Server*).

Resumindo, este trabalho deu-nos a preparação prática necessária para criar soluções Full-Stack modernas e fiáveis.